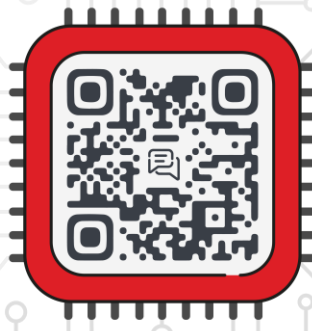




نظري

5

15

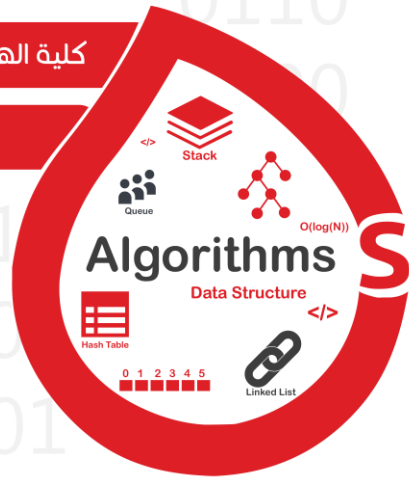
1800 SP

كلية الهندسة المعلوماتية

السنة الثانية

Divide and Conquer Algorithms

د. أبي صندوق



RB Informatics; 09/09/2020

خوارزميات 1

كنا قد تحدثنا في المحاضرات السابقة عن طرق التفكير و تناولنا طريقة Decrease and Conquer، و اليوم عدنا لنتابع رحلتنا مع طريقة تفكير جديدة هي Divide and Conquer، كونوا مستعدين.

Decrease and Conquer: التقليل و التغلب.

Divide and Conquer: التقسيم و التغلب.

ما هو الـ Divide and Conquer ؟؟؟

هو أسلوب لحل المشكلات يعتمد على فكرة ذكية كالآتي:
إذا كانت المشكلة كبيرة و معقدة
← نقسمها إلى مشاكل أصغر و أسهل
← نحل كل واحدة منهم
← ندمج الحلول لنحصل على حلّ المشكلة الأصليّة

مثال تمهيدي:

نفترض أنّ لدينا سلسلتين مرتبتين تصاعدياً
السلسلة الأولى: 3 5 8 12 عناصر $n = 4$
السلسلة الثانية: 1 7 9 13 عناصر $n = 4$
و نهدف إلى دمج السلسلتين للحصول على سلسلة واحدة مرتبة تصاعدياً.

الطريقة 1:

نضع السلسلتين إلى جانب بعضهم البعض لنحصل على سلسلة واحدة بالشكل:
عناصر $n = 8$ 3 5 8 12 1 7 9 13

الطريقة 2:

نختار العنصر الأصغر في كل من السلسلتين و نضعه في السلسلة الجديدة (التي نصنعها)

السلسلة 1

3	5	8	12
---	---	---	----

السلسلة 2

1	7	9	13
---	---	---	----

السلسلة الجديدة

1	
---	--

و نكرر العملية ذاتها:

السلسلة 1

3	5	8	12
---	---	---	----

السلسلة 2

7	9	13
---	---	----

نختار الأصغر بينهما (و هو 3):

السلسلة الجديدة

1	3	
---	---	--

و تستمر هذه العملية حتى تنتهي إحدى السلسلتين عندئذٍ فإن بقيّة العناصر في السلسلة الأخرى (التي لم تنته بعد) توضع كما هي في السلسلة الجديدة، و هكذا نكون قد حصلنا على سلسلة مرتبة تصاعدياً من سلسلتين مرتبتين تصاعدياً.

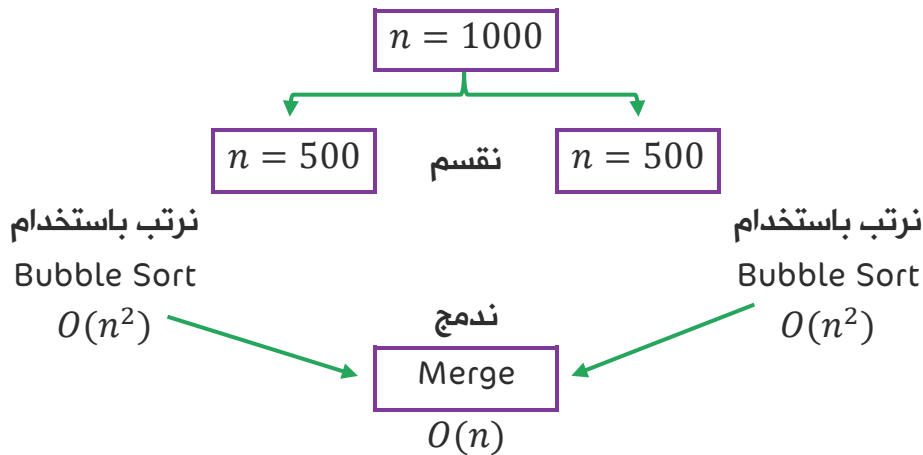
مثال:

نريد ترتيب سلسلة تحوي أرقاماً عشوائية فيها $n = 1000$ عنصراً

- إذا قمت بترتيبها بطريقة Bubble Sort أو Insertion Sort و التي تعقيدها $O(n^2)$ ، هذا يعني أنك بحاجة إلى 1,000,000 (op).
- لنفكر بطريقة تساعدنا في اختصار الوقت..



مثلاً نقسم السلسلة إلى قسمين متساويين:



و باستخدام Bubble Sort نريد ترتيب السلسلة:

- الترتيب: $n = 1000$ عنصراً $\Leftarrow$ نحتاج تقريباً $(1000)^2 \approx 1\,000\,000$ (op)
- لكن عند تقسيم المسألة سنرتب $n = 500$ عنصراً $\Leftarrow$ سنحتاج تقريباً: $(500)^2 + (500)^2 = 500\,000$ (op)
- من أجل عملية الدمج نستخدم Linear Merge بتعقيد خطي $O(n)$ أي سنحتاج (op) 1000 من أجل الدمج.

من خلال عملية التقسيم هذه أكون قد وفّرت 49% تقريباً من الوقت بالرغم من أنني أنفذ خوارزمية Bubble Sort ذاتها.

كيف استطعت توفير الوقت؟؟؟

نعلم أنّ تعقيد Bubble Sort $\Leftarrow$ تربيعي $O(n^2)$

و تعقيد عملية الدمج $\Leftarrow$ خطّي $O(n)$

إذاً ففي مرحلة الدمج أكون قد كسبت الفرق بين التربيعي و الخطي عندما قسمت السلسلة إلى نصفين.

و فيما لو قسمت السلسلة إلى ثلاثة أقسام فإنني سأوفّر وقت أكثر لأنّ الجزء الذي تعقيده تربيعي (و هو عملية Bubble Sort على الأجزاء) يصبح أصغر، بينما الجزء الذي تعقيده خطّي (عملية الدمج) يصبح أكبر، و بسبب الفرق بين تعقيد العمليتين أكسب الوقت.

ملاحظة:

يمكن استخدام خوارزمية Insertion Sort بدلاً من Bubble Sort لأنّ كليهما تقوم بنفس العمل و بتعقيد $O(n^2)$.

إذاً نلاحظ أن:

$$(300)^2 + (300)^2 + (400)^2 < (500)^2 + (500)^2 < (1000)^2$$

دون تقسيم التقسيم إلى نصفين التقسيم إلى ثلاثة أقسام

نستنتج أنه عند تجزئة المسألة أكون قد استفدت و ربحت وقت طالما لدي طريقة لدمج الأجزاء.

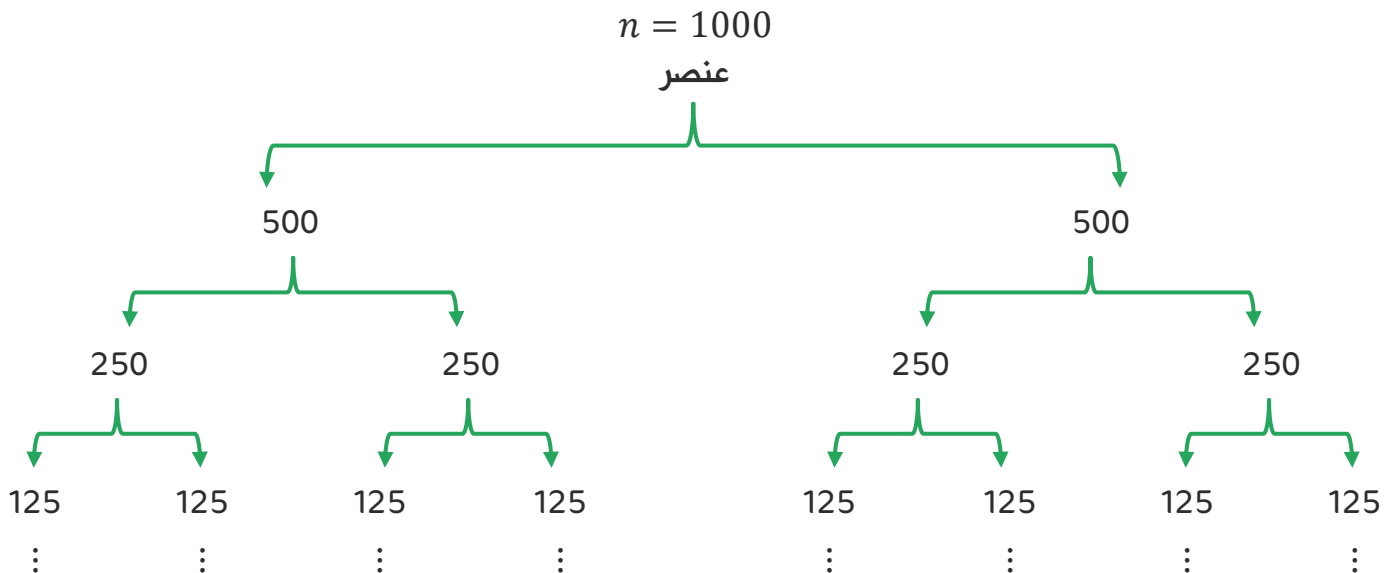
"و إذا ما عندي طريقة ادمج الأجزاء بعد التقسيم ؟!"
هون من الأساس لا تقسم لأنك ما بتستفيد شي بهالحالة.

➤ هذه الخوارزمية تسمى Merge Sort

آلية عمل الخوارزمية:

1. تقسيم المصفوفة إلى نصفين.
2. ترتيب كل نصف.
3. دمج النصفين المرتبين.

➤ مثلاً:



لماذا الطريقة العودية هي أفضل لكتابة هذه الخوارزمية ؟

هنا يجب أن يلفت نظرنا شيء مهم... وهو أن الخطوة الثانية هي تكرار لنفس المسألة الأصلية بالضبط، حيث إنه لترتيب سلسلة باستخدام هذه الخوارزمية تحتاج لترتيب سلسلتين أصغر بنفس الخطوات تماماً. وهذا هو جوهر الاستدعاء الذاتي الذي يقوم على تكرار نفس الخطوات في كل استدعاء.

ملاحظة:

- يمكن تطبيق الخوارزمية من غير الاستدعاء الذاتي و لكن سيكون الكود معقد جداً.
- المسائل ذات طبيعة Divide and Conquer تكتب بالطريقة العودية.

كيف تتم عملية الدمج ؟

تتم باستخدام خوارزمية بسيطة تدعى Linear Merge تستخدم لدمج مصفوفتين مرتبتين في مصفوفة واحدة مرتبة بتعقيد خطي $O(n)$.

مثال: $A = [2,5,8]$ $B = [3,4,9]$

نبدأ من أول عنصر في كل مصفوفة ونقارن $2 < 3$

و نضع العنصر الأصغر في المصفوفة الناتجة و العنصر الذي يُضاف للمصفوفة الناتجة يُحذف من مصفوفته الأصلية.

2	
---	--

ثم نقارن مرة أخرى من أجل أول عنصرين $3 < 5$

2	3	
---	---	--

و هكذا حتى تنتهي إحدى المصفوفات ثم نضيف ما تبقى من المصفوفة الأخرى.

الكود:

```
1 void MergeSort (int[] A, int st, int ed) {
2     // نقطة البداية
3     // ed; نقطة النهاية
4     if (ed - st <= 1) return;
5     // شرط التوقف
6     int m = (st + ed) / 2;
7     // m: نقطة المنتصف
8     MergeSort(A, st, m); // سيضمن أن القسم الأيسر مرتب
9     MergeSort(A, m + 1, ed); // سيضمن أن القسم الأيمن ترتب
10    LinearMerge(A, st, ed, m); // الدمج بين القسمين
11 }
```

إنشاء تابع Linear Merge:

```
1 public static int[] linearMerge(int[] arr1, int[] arr2) {
2     int n1 = arr1.length; // حجم المصفوفة الأولى
3     int n2 = arr2.length; // حجم المصفوفة الثانية
4     int[] result = new int[n1 + n2]; // مصفوفة النتيجة
5     int i = 0; // مؤشر المصفوفة الأولى
```

```

6   int j = 0; // مؤشر المصفوفة الثانية
7   int k = 0; // مؤشر مصفوفة النتيجة
8   while (i < n1 && j < n2) {
9       // مقارنة العناصر الحالية من المصفوفتين
10      if (arr1[i] < arr2[j]) {
11          result[k] = arr1[i];
12          i++;
13      } else {
14          result[k] = arr2[j];
15          j++;
16      }
17      k++;
18  }
19  // إضافة العناصر المتبقية من المصفوفة الأولى
20  while (i < n1) {
21      result[k] = arr1[i];
22      i++;
23      k++;
24  }
25  // إضافة العناصر المتبقية من المصفوفة الثانية
26  while (j < n2) {
27      result[k] = arr2[j];
28      j++;
29      k++;
30  }
31  return result;
32  }

```

و لإيجاد تعقيد خوارزمتنا فإننا نتبع الطريقة المعروفة:

السطر	Cost	Time
4	1	1
6	1	1
8	$T\left(\frac{n}{2}\right)$	1
9	$T\left(\frac{n}{2}\right)$	1
10	$O(n)$	1

نلاحظ في السطرين الثالث و الرابع أن التكلفة هي $T\left(\frac{n}{2}\right)$ ، لأننا هنا لا نقوم بعملية جمع، ضرب... فالتكلفة ليست عدداً معيناً من العمليات، نحن هنا أمام استدعاءات للتتابع و التي تكلف مقداراً من الزمن اللازم لتنفيذ الخوارزمية على $\frac{n}{2}$ عنصر، أي $T\left(\frac{n}{2}\right)$.
و في السطر العاشر التكلفة من رتبة $O(n)$.

إذاً لحساب التعقيد الزمني:

تابع التعقيد الزمني: $T(n) = Cost \times Time$

و في مثالنا: $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

$T\left(\frac{n}{2}\right)$: الزمن اللازم لترتيب نصف العناصر.

$\frac{T(n)}{2}$: نصف الزمن اللازم لترتيب n عنصر.

$$T\left(\frac{n}{2}\right) \neq \frac{T(n)}{2} \quad \text{انتبه !!!}$$

و التابع هنا معرّف بدلالة نفسه "تابع رياضي عودي" لكنه غير قابل للمقارنة مع غيره $\Leftarrow$ و السبب: هو أن الاستدعاء العودي

غير واضح الرتبة "أي لا يمكنني تحديد الصف الذي ينتمي له".

بينما العمليات الواضحة مثل: "العداد - حلقة - while - return (...)

جميعها عمليات واضحة من الرتبة 1.

نحن هنا أمام مشكلة في عدم القدرة على تحديد الصف الذي ينتمي إليه التابع !!

تذكرة: إذا كان لدينا التابع: $n^2 + 5n + 3$ فهو ينتمي للصف n^2 "لمعرفة الصف نختار الأس الأعلى و نهمل باقي الحدود".

و لمعرفة الصف الذي ينتمي له التابع العودي نستخدم طريقة Master Theorem و هي طريقة (رياضيات بحت)

تعطينا مقابل للتابع العودي بشكل صريح نوجد من خلاله الصف المناسب.

"يعني باختصار بعطيها التابع العودي بتقلّي هالتابع من الصف كذا".

لاستخدام طريقة Master Theorem يجب أن يكون شكل التابع العودي هو:

$$a \cdot T\left(\frac{n}{b}\right) + O(n^k)$$

a: تمثل عدد الأجزاء التي تتم معالجتها.

b: حجم الجزء (المسألة تقسم على b جزء).

$O(n^k)$: كلفة الدمج و يعبر k عن رتبة تعقيد دمج الحلول.

⚡ و نميز ثلاث حالات:

$$1. a > B^k$$

التعقيد الحقيقي سببه عمليات التقسيم / التجزئة (Partition) و هنا يكون تعقيد عملية الدمج مهمل أمام تعقيد عمليّة

تجزئة المسائل {أي يتم تقسيم المسألة لعدد كبير من المسائل فيكون مكلفاً}.

$T(n) \in O(n^{\log_B a})$ الصف المناسب:

$$2. a < B^k$$

التعقيد الحقيقي سببه عملية الدمج و هنا يكون تعقيد عملية التجزئة مهماً "لأنه بطريقة ما كانت التجزئة سريعة جداً بالنسبة لعملية دمج مكلفة أكثر".

$$\leftarrow \text{الصف المناسب: } T(n) \in O(n^k)$$

$$3. a = B^k$$

كتلة التعقيد بسبب التجزئة تساوي تقريباً كتلة التعقيد بسبب الدمج "هنا تتساوى الأوزان في التعقيد".

$$\leftarrow \text{الصف المناسب: } T(n) \in O(n^k \cdot \log_B(n))$$

آلية عمل Master Theorem

تأخذ المجموعة الأساسية التي لديك و المكوّنة من n عنصر و تقسمها لمجموعة مسائل جزئية، ثمّ تحلّ مجموعة من هذه المسائل الجزئية و ليس بالضرورة أن تحلّها كلها.

فمثلاً، قد تقسم إلى 10 مسائل جزئية و تحلّ 3 مسائل فقط و تهمل البقية لسبب ما، أو قد تقسم إلى 8 مسائل جزئية و لكنها تمرّ على كل مسألة جزئية مرتين فهي فعلياً تحلّ 16 مسألة... و هكذا...
إذاً فحسب كيفية تقسيم الـ Data التي لديك و عدد المسائل التي تحلّها تعطيك قيم a و B ، و يظهر أيضاً تابع إضافي من أجل عملية الدمج قد تكون تكلفته $O(1)$ أي $k = 0$ و هنا عملية الدمج مهمة...

أو قد تكون تكلفته $O(n)$ أي $k = 1$ كما في مثالنا السابق...

أو قد تكون تكلفته $O(n^2)$ بحسب كلفة عملية الدمج...

بالعودة إلى مثالنا نجد أنّ شكل التابع يماثل الشكل المطلوب.

$$T(n) = 2 T\left(\frac{n}{2}\right) + O(n^1)$$

$a = 2$ $B = 2$ $k = 1$

و بالمقارنة:

$$a = B^k$$

$$2 = 2^1$$

إذاً نحن أمام الحالة الثالثة بحسب Master Theorem و الصف المناسب هو:

$$T(n) \in (n \cdot \log n)$$

لنجرّب كمثال تابع البحث الثنائي (Binary Search) بتطبيق الـ Master Theorem :

البحث الثنائي يتطلب منا أن نحلّ بنصف عدد العناصر $T\left(\frac{n}{2}\right) \leftarrow$
بالإضافة إلى بعض العمليات البسيطة $\leftarrow$ ثابت

فإن التابع الزمني هو: ثابت $T(n) = T\left(\frac{n}{2}\right) +$

و بالمقارنة:



$$\begin{aligned} a &= 1 \\ B &= 2 \\ k &= 0 \end{aligned}$$

$$\begin{aligned} a &= B^0 \quad \leftarrow \\ 1 &= 2^0 \end{aligned}$$

نحن أمام الحالة الثالثة بحسب Master Theorem

$$\begin{aligned} \Rightarrow T(n) &\in O(n^0 \cdot \log n) \\ \Rightarrow T(n) &\in O(\log n) \end{aligned}$$

و هذا صحيح لأن تعقيد تابع الـ Binary Search من الصف $\log n$

خوارزمية Quick Sort

هي خوارزمية تستخدم لترتيب عناصر في قائمة و هي من أسرع خوارزميات الترتيب.

كيف تعمل خوارزمية "Quick Sort" ؟

- نختار رقم عشوائي pivot (عنصر محوري) و ليكن أول عنصر مثلاً.
- و نحدّد مؤشرين، المؤشر الأول على بداية المصفوفة و المؤشر الثاني عند نهايتها (آخر عنصر).
- لنبدأ بالمؤشر الأول الذي يمرّ على عناصر المصفوفة حتى يجد عنصر قيمته أكبر من key يتوقف عند هذا العنصر.
- و يتحرك المؤشر الثاني من نهاية المصفوفة فيمرّ على العناصر حتى يجد عنصر قيمته أصغر من key فيتوقف عند هذا العنصر.
- نبادل بين العنصرين الذين توقّفنا عندهما (swap) و نستمر بهذه العملية حتى نحصل على

أكبر من pivot | pivot | أصغر من pivot

ثم نكرّر العملية ذاتها من أجل كل جزء حتى الوصول إلى مصفوفة مرتبة بالكامل.

نفرض لدينا المصفوفة الآتية :

A	7	3	15	5	10	12	8	9	13	1	4
---	---	---	----	---	----	----	---	---	----	---	---

1. نأخذ عنصر عشوائي وليكن الأول مثلاً ، ونضع مؤشر في البداية ومؤشر في النهاية .

A	7	3	15	5	10	12	8	9	13	1	4
	Pivot	i								j	

(عنصر محوري)

2. نستمر بتحريك المؤشر (i) حتى نجد عنصر أكبر من 7 فنجد $7 < 15$ ثم نستمر بتحريك المؤشر (j) حتى نجد عنصر أصغر من 7 فنجد $7 > 4$

A	7	3	15	5	10	12	8	9	13	1	4
		i								j	

نبدل بينهما

A	7	3	4	5	10	12	8	9	13	1	15
		i								j	

3. نكرر العملية فنجد

▪ عند تحريك (i) $7 < 10$ ▪ وعند تحريك (j) $7 > 1$

A	7	3	4	5	10	12	8	9	13	1	15
					i					j	

نبدل بينهما

A	7	3	4	5	1	12	8	9	13	10	15
					i					j	

4. نكرر العملية فنجد:

▪ عند تحريك (i) $7 < 12$

▪ وعند تحريك (j) أن (j) قد تجاوز (i) وهذا يعني أن المكان الواقف j عنده

كل ما قبله أصغر من 7 ، وكل ما بعده أكبر من 7

A	7	3	4	5	1	12	8	9	13	10	15
					j	i					

لذا نبذل الآن بين الـ (7) و $A[j]$

A	1	3	4	5	7	12	8	9	13	10	15
	نرتبها					نرتبها					

هكذا نكون أوجدنا المكان الصحيح للعنصر 7 و لإكمال ترتيب المصفوفة نعيد الخطوات على كل جزء أو نرتب كل جزء على حدى.

الكود:

```

1 public class QuickSort {
2     public static void quickSort(int[] A, int left, int right){
3         if (left < right) {
4             int pivotIndex = partition(A, left, right);
5             // sort left part
6             quickSort(A, left, pivotIndex - 1);
7             // sort right part
8             quickSort(A, pivotIndex + 1, right);
9         }
10    }
11
12    private static int partition(int[] A, int left, int right){
13        int pivot = A[left];    // pivot = first element
14        int i = left + 1;
15        int j = right;
16        while (i <= j) {
17            // move i to the right
18            while (i <= right && A[i] <= pivot) {
19                i++;
20            }
21            // move j to the left
22            while (A[j] > pivot) {
23                j--;
24            }
25            // swap A[i] and A[j]
26            if (i < j) {
27                int temp = A[i];
28                A[i] = A[j];
29                A[j] = temp;
30            }
31        }
32        // place pivot in its correct position
33        int temp = A[left];
34        A[left] = A[j];
35        A[j] = temp;
36        return j; // pivot final position
37    }

```

```

38 // Test
39 public static void main(String[] args) {
40     int[] A = {7, 3, 15, 5, 10, 12, 8, 9, 13, 1, 4};
41
42     quickSort(A, 0, A.length - 1);
43
44     for (int x : A) {
45         System.out.print(x + " ");
46     }
47 }
48 }

```

- خوارزمية الـ "Quick Sort" من أفضل الخوارزميات لأنها تترتب من تلقاء نفسها في مرحلة التقسيم دون الحاجة لعملية دمج.
- تعقيدها يكون بين $O(n \log n)$ و $O(n^2)$.
- في الحالة المتوسطة فإن تعقيدها يكون أقرب لـ $O(n \log n)$ و ميزة هذه الخوارزمية أنها لا تحوي الكثير من الاستدعاءات العودية لذلك هي فعالة أكثر، لأن الاستدعاء العودي لا يغير التعقيد لكنه يأخذ وقتاً أطول قليلاً.
- لذلك فإن "Quick Sort" معتمدة في المكتبات البرمجية أكثر من "Merge Sort".

ملاحظات على Quick Sort:

1. إن أفضل حالة هو أن يكون المكان الصحيح لـ Pivot هو منتصف المصفوفة.
 2. إن أسوأ حالة هو أن يكون الـ Pivot هو أصغر / أكبر عنصر في المصفوفة، لأنني سأنفذ الترتيب على n عنصر $\Leftarrow$ التعقيد: $O(n^2)$.
 3. ما هو الرقم العشوائي (Pivot) كيف سأختاره ؟
- بدون شروط و غالباً نفترض أن (Pivot) هو أول عنصر في المصفوفة، **لكن** انتبه: إذا كانت المصفوفة شبه مرتبة فإن اختيار خوارزمية Quick Sort هو أساساً اختيار سيئ و الأفضل اختيار Insertion Sort.

خوارزمية K-Select (Quick Select)

هي خوارزمية اختيار انتقائية لإيجاد عنصر ذو رتبة محددة في مصفوفة غير مرتبة، و هي تعتمد على مبدأ تقسيم المصفوفة المشابه لخوارزمية "Quick Sort".

⚡ كيف تعمل خوارزمية (K-Select) ؟؟؟

1. اختيار محور (المحور هو أي عنصر من المصفوفة نبحث على أساسه).
2. تقسيم المصفوفة (ترتيب العناصر بحيث تتوضع العناصر الأصغر من المحور على يساره و العناصر الأكبر من المحور على يمينه).
3. اختيار الجزء المناسب (في أي قسم من المصفوفة نوجد العنصر المطلوب، أي على يمين أم يسار المحور).
4. تكرار الخطوات حتى نحصل على العنصر المطلوب.

مثال:

لدينا المصفوفة [12, 5, 9, 2, 15, 7, 3]

هدفنا هو إيجاد رابع أصغر عنصر في المصفوفة $K = 4$.

خطوات الحل:

1. نختار المحور

المحور = 3 (غالباً نختار آخر عنصر كمحور)

2. تقسيم المصفوفة حول المحور

← كل العناصر التي تكون أصغر أو تساوي 3 على اليسار

← كل العناصر التي تكون أكبر من 3 على اليمين

نمرّ على عناصر المصفوفة و نقارن مع (المحور = 3)

فتصبح المصفوفة من الشكل:

ملاحظة: لا نرتب المصفوفة، بل نضع المحور في مكانه المناسب.

3. تحديد موقع الـ index الذي يقع فيه المحور

3 موجود في الـ index = 1

أي يوجد عنصران أصغر أو يساوي 3.

4. المقارنة مع المطلوب (الترتيب هو $K = 4$)

← العناصر الأصغر من المحور هي عنصرين

و بما أننا نبحث عن رابع أصغر عنصر، إذاً فالجزء اليساري (قبل المحور) لا يخدمنا و سأنقل للبحث في الجزء اليميني (بعد المحور)

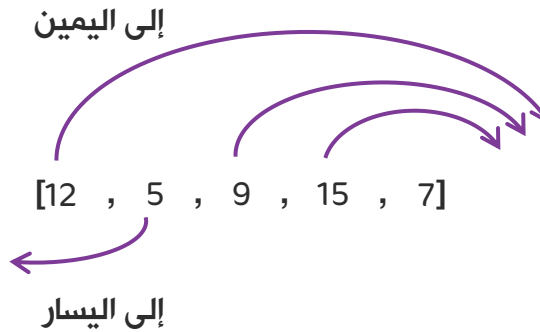
والجزء اليميني الذي يحتوي على العناصر الأكبر من المحور = 3:

[12, 5, 9, 15, 7]

1. نختار محور جديد

(المحور = 7) في المصفوفة الجزئية

2. تقسيم المصفوفة الجزئية و وضع عنصر المحور 7 في مكانه المناسب



فتصبح:

[5, 7, 12, 9, 15]

وهنا يقع (المحور = 7) في الـ index = 1 من المصفوفة الجزئية

3. حساب الرتبة:

أ. نحسب الـ index الأصلي للمحور الحالي 7 بالنسبة للمصفوفة الأصلية و بعد التقسيم الثاني.

← نجد أن index الـ 7 هو 3

ب. حساب الرتبة:

الـ index الأصلي + عدد العناصر قبل المحور (في مثالنا قبل الـ 7 يوجد عنصر واحد فقط هو 5)

رتبة 7 هي $3 + 1 = 4$ ، إذاً فإن 7 هو رابع أصغر عنصر في المصفوفة.

الكود:

```
1 public class k-Select {
2     public static int kSelect(int[] arr, int low, int high, int k){
3         if (low <= high) {
4             int p = partition(arr, low, high);
5             if (p == k - 1)
6                 return arr[p];
7             else if (p > k - 1)
8                 return kSelect(arr, low, p - 1, k);
9             else
10                return kSelect(arr, p + 1, high, k);
11        }
12        return -1; // invalid case
13    }
```

```
14 private static int partition(int[] arr, int low, int high) {
15     int pivot = arr[high];
16     int i = low - 1;
17     for (int j = low; j < high; j++) {
18         if (arr[j] <= pivot) {
19             i++;
20             swap(arr, i, j);
21         }
22     }
23     swap(arr, i + 1, high);
24     return i + 1;
25 }
26 private static void swap(int[] arr, int i, int j) {
27     int temp = arr[i];
28     arr[i] = arr[j];
29     arr[j] = temp;
30 }
31 public static void main(String[] args) {
32     int[] arr = {6, 8, 3, 5, 2, 4, 1, 7};
33     int k = 4;
34     int result = quickSelect(arr, 0, arr.length - 1, k);
35     System.out.println(k + "the smallest element = " + result);
36 }
37 }
```

اللهم بارك لنا السَّعي بالقبول

و ارزقنا لذة الوصول

